

Manual de instalación

Cómo levantar el tablero, solo o junto con la API de inferencia.

El producto son dos piezas en dos repositorios:

Repositorio	Contiene
maia-pds-microproyecto-api	La API, el modelo empaquetado y docker-compose.yml
maia-pds-microproyecto-ui (este)	El tablero

El tablero **no funciona solo**: sin una API que responda, las tres vistas cargan pero no hay predicciones. Por eso la opción recomendada levanta las dos.

Opción A — los dos servicios con Docker Compose (recomendada)

Es la instalación completa del producto y la más corta.

Requisitos: Docker con Docker Compose v2 (`docker compose version`).

```
git clone https://github.com/katterine2558/maia-pds-microproyecto-api.git
cd maia-pds-microproyecto-api
docker compose up --build
```

El docker-compose.yml construye la API desde ese repositorio y el tablero directamente desde este, sin necesidad de clonarlo.

Cuando termine:

- Tablero: <http://localhost:8501>
- API: <http://localhost:8000/docs>

Verificación:

```
curl -s http://localhost:8000/health
# {"estado":"ok", "modelo":"bosque_formulario_e3_v1"}
```

Y en el tablero, vista **Paciente** → **Calcular riesgo** debe devolver una probabilidad.

Para detenerlo: `docker compose down`.

El tablero espera a que la API esté *healthy*, no solo arrancada: `/health` responde 503 mientras el modelo no esté cargado. Por eso el primer arranque tarda unos segundos más.

Opción B — solo el tablero, en contenedor

Útil cuando la API ya está corriendo en otra parte.

```
git clone https://github.com/katterine2558/maia-pds-microproyecto-ui.git
cd maia-pds-microproyecto-ui
docker build -t reingreso-ui .
docker run --rm -p 8501:8501 -e API_URL=http://host.docker.internal:8000 reingreso-ui
```

API_URL es la única configuración. Apunta a donde esté la API:

Dónde corre la API	Valor de API_URL
En la misma máquina, fuera de Docker	<code>http://host.docker.internal:8000</code>
En la misma red de Compose	<code>http://api:8000</code>
Desplegada con dominio público	<code>https://<dominio-de-la-api></code>

Dónde corre la API	Valor de API_URL
En Railway, por la red privada	<code>http://<servicio-api>.railway.internal:<puerto></code>

Sin barra al final: el tablero la agrega.

Opción C — el tablero en local, sin Docker

Para desarrollar sobre el tablero.

Requisitos: Git, Python 3.12 y uv.

```
git clone https://github.com/katterine2558/maia-pds-microproyecto-ui.git
cd maia-pds-microproyecto-ui
uv sync
API_URL=http://localhost:8000 uv run streamlit run app.py
```

Queda en <http://localhost:8501>.

`services/api.py` lee `API_URL` **del entorno**. Copiar `.env.example` a `.env` no la carga por sí solo: expórtela en la sesión o pásela en la línea de comando, como arriba.

Para levantar la API por separado, desde el repositorio `-api`:

```
uv run --no-project --with-requirements api/requirements.txt \
  uvicorn api.main:app --port 8000
```

FastAPI no está en las dependencias base de ese repositorio: la API trae su propia lista en `api/requirements.txt`, y por eso el comando la pide aparte.

Desplegar en Railway

Ambos servicios se despliegan desde sus Dockerfile, en el mismo proyecto de Railway. El tablero tiene dominio público; la API no: el tablero la alcanza por la red privada del proyecto. El procedimiento completo está en `docs/soportes/despliegue-railway.md` del repositorio de la API.

Lo esencial para el tablero:

1. **New Project → Deploy from GitHub repo** → `maia-pds-microproyecto-ui`.
2. Rama `develop`. Railway reconstruye sola en cada push.
3. **Settings → Networking → Generate Domain**.
4. Agregar la variable `API_URL` con la dirección privada de la API: `http://<servicio-api>.railway.internal:<puerto>`, donde `<servicio-api>` es el nombre del servicio de la API en Railway y `<puerto>` el que aparece en sus logs (`Uvicorn running on http://[::]:<puerto>`). La red privada de Railway es IPv6; el Dockerfile de la API ya escucha en `::`.

Railway inyecta su propia variable `PORT` en tiempo de ejecución y pisa el `ENV PORT` de la imagen. Por eso el CMD usa `${PORT:-8501}`, y el puerto destino del dominio es el que Streamlit anuncia en los logs del despliegue (URL: `http://0.0.0.0:<puerto>`), no el del EXPOSE.

Problemas frecuentes

El botón Calcular riesgo devuelve error de conexión. `API_URL` no apunta a una API que responda. Compruébelo desde la misma máquina donde corre el tablero:

```
curl -s "$API_URL/health"
```

Dentro de un contenedor, `localhost` es el propio contenedor, no el anfitrión: use `host.docker.internal` o el nombre del servicio de Compose.

docker compose up falla al construir el tablero. El context del servicio tablero apunta al repositorio de GitHub y necesita red. Si está trabajando con una copia local, cámbielo por context: ../maia-pds-microproyecto-ui (viene comentado en el archivo).

El puerto 8501 está ocupado. Cambie el mapeo: -p 8600:8501, o el ports del docker-compose.yml.

La API responde 503. El modelo no está cargado. Compruebe que api/artifacts/modelo.joblib viajó dentro de la imagen; está versionado en Git, así que no debería faltar.